

Projektbeskrivning

Gameboy Emulator

2025-02-19

Projektmedlemmar:

Natalie Holmström <natho280@student.liu.se>

Fritiof Santesson <frisa670@student.liu.se>

Handledare:

Stina Åström <stias606@student.liu.se>

Innehåll

1. Introduktion till projektet.....	2
2. Ytterligare bakgrundsinformation.....	2
3. Milstolpar.....	2
4. Övriga implementationsförberedelser.....	4
5. Utveckling och samarbete.....	4
6. Implementationsbeskrivning.....	6
6.1. Milstolpar.....	6
6.2. Dokumentation för programstruktur, med UML-diagram.....	6
7. Användarmanual.....	7

Projektplan

1. Introduktion till projektet

Vi ska göra en gameboy-emulator som ska kunna läsa in en binär fil (ROM-fil) innehållande ett Game Boy-spel och låta användaren spela det.

I grova drag består en emulator av en modell av den riktiga hårdvaran, avläsning av binära instruktioner från fil, och tolkning av de binära instruktionerna för att manipulera hårdvaran. Till sist presenteras displayen och knappar att trycka på till användaren – precis som på den riktiga konsolen.

Vi kommer också sikta på att implementera debugging-tillägg som möjligheten att inspektera minnet, och eventuellt dekompilering (egentligen bara en lista på de instruktioner som finns i programmet). Vi lämnar utrymme för att det sistnämnda kan vara mycket svårare än vi förutser, men vi vill i alla fall ha det i åtanke. Detta för att inte snöa in helt på att skriva själva emulatorn, vilket kan vara svårt att hålla “rent objektorienterat”, och vill därför ha några extra features som lämpar sig bättre för att visa upp våra kunskaper i Java.

2. Ytterligare bakgrundsinformation

För att skriva en emulator krävs mycket information om hårdvarans funktionalitet, och även hur instruktionerna ser ut och fungerar. I teorin kan man hitta allt detta genom reverse engineering (demontering), men med tanke på all information som går att hitta online bortser vi från detta då vi anser det som överkomplicerat och även onödigt. Vi använder oss (i alla fall inledningsvis) av följande webbsidor, som kompletteras av diverse mer specifika google-sökningar o.d.

<https://meganesu.github.io/generate-gb-opcodes/>

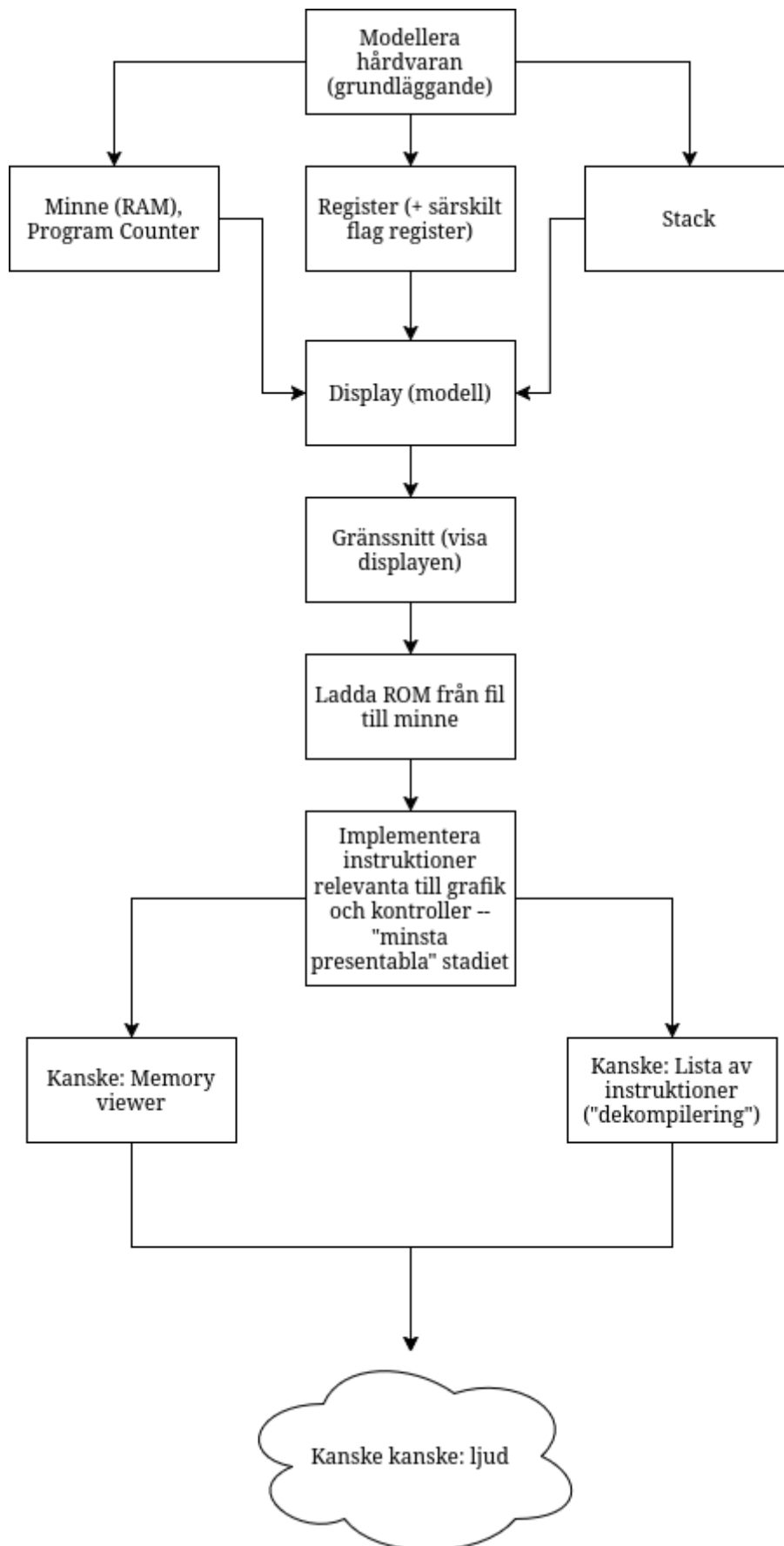
<https://gbdev.io/pandocs/>

Länkarna ovanför ger en överblick av hårdvaran och instruktionerna vi behöver implementera.

Sist men inte minst kommer vi mest troligt att ignorera vissa egenskaper hos gameboy-hårdvaran fullständigt: det finns en IR-port, länkkabel, och självklart ljud och musik. De två första är ingenting som krävs för att kunna spela 99% av alla spelen, så vi siktar inte ens på att genomföra dem.

Ljud är något som vi klarar oss utan för att kunna visa upp projektet, och är lätt att “strunta i” medan man implementerar emulatorn. Så, om vi inte skulle finna oss med massor och åter massor av extra tid, är det inget vi kommer försöka oss på.

3. Milstolpar



#	Beskrivning
1	Modell av hårdvara (flera steg):
2	RAM: val av datatyp och struktur
3	RAM: Hur många bitar på varje plats?
4	RAM: Hur många bitar i adressen?
5	RAM: Fundera över lämpliga metoder vi vet med oss kommer behövas (detta KOMMER vi behöva återgå till senare)
6	Register: hur många, vilken datatyp?
7	Register: Metoder, setter/getters, hur ska vi “snyggt” komma åt enskilda register?
8	Stack: Vart ligger den, hur fungerar den – ska man kunna spara godtyckliga värden eller bara värdet av PC vid hopp? Implementera.
9	Display: Storlek i pixlar samt storlek i verkligheten (dvs. vad har den för förhållande mellan bredd:höjd)
10	Display: Implementera grundläggande färger
11	Display: Put/get pixel, grundläggande funktionalitet – vi KOMMER behöva återgå hit när vi vet hur gameboy:en faktiskt ritar på skärmen i detalj
12	Gränssnitt: MINIMALT fungerande gränssnitt för att rita ut skärmen för oss medans vi utvecklar
13	Ladda ROM från fil: läs in bytes och placera
14	ROM: Ska ha en sektion (bank) som alltid är läsbar, och X antal banker som programmet kan byta mellan och läsa
15	ROM: Fundera över hur detta implementeras – kommer förmodligen behöva återgå hit när vi har en bättre bild av hur det sker i praktiken, “vad programmen förväntar sig” utav emulatorn
16	Implementera instruktioner: Den här punkten är omöjlig att dela upp här och kommer ta den allra största delen av utvecklingen. Räkna med att det här steget egentligen tar upp säkert lika många steg till, som vi redan har. Ingen enskild lär ta överdrivet lång tid, men det finns många, och de ska fungera på väldigt specifika sätt.
17	Eventuellt: minnesinspektion; en lista (sökbar?) där vi kan slå upp olika minnesadresser och se vad som finns på dem.
18	Minnesinspektion: Implementera den logiskt. Hur ofta ska den uppdateras,

live eller vid paus av exekvering? (Det första kommer att ta prestanda – märkbart?)

19 Minnesinspektion: Visa upp minnesinspektionen i gränssnittet. Samma fönster som displayen, eller separat?

20 Dekompilering: Översätt alla maskininstruktioner till antingen det riktiga assemblyspråket för processorn (svårt) eller pseudo-assembly som ett proof-of-concept att visa upp.

21 Dekompilering: Visa dekompileringen i gränssnittet. Eventuellt i samma fönster som displayen.

22 Pie in the sky: implementera ljud, börja med hårdvaran

23 Ljud: Spela upp ljudet i gränssnittet.

...

4. Övriga implementationsförberedelser

I stort har detta redan beskrivits ovan, men ett generellt mål är att tänka objektorienterat: en emulator är ett program som inte “faller sig naturlig” att dela upp objektorienterat, så att fundera noga över lämpliga objektorienterade abstraktioner är något vi vill lägga fokus på.

I stort kommer vi ha en main-funktion som startar ett gränssnitt och själva emulator-klassen. Emulator-klassen kommer i sin tur ha fält för register, minne, display, etc, som är separata klasser. ROM-filen läser programmen av som vanliga minnesadresser, och därför kommer ROM-filen också vara ett fält som tillhör minnes-klassen.

5. Utveckling och samarbete

Vi har samma ambitionsnivå. Vi siktar båda på betyg 5, men har också valt ett svårt projekt med sikte på att lära oss kanske lite extra!

Vi är flexibla och jobbar ihop när tid finns och efter behov. Det bör nämnas att vi labbar ihop i 3 kurser, så *vad* vi jobbar med varje labbtillfälle bestäms oftast efter vad vi känner är mest brådskande för stunden. Vi har inga fasta “schemalagda” tider, utan bestämmer oftast när vi ska labba igen i slutet av varje pass. Vi jobbar under veckorna, men även på helger (lite efter behov). Resurstillfällena kommer vi förmodligen bara gå på om vi stöter på väldigt specifika problem, rent Java-mässiga eller relaterade till IntelliJ. (Vi vet inte hur mycket det är okej att fråga om på detaljnivå ang. själva emulatorn?)

Det är väldigt svårt att hitta en milstolpe som vi ska ha klarat vid ett visst datum, eftersom att implementationen av alla instruktioner (400-450 st) kommer att vara större delen av projektet (där vissa är extremt enkla, och andra är riktigt luriga).

Godtagbara skäl för att inte dyka upp på bestämda tider är oförutsedda hinder (sjukdom, dödsfall inom närmsta kretsen...) – så länge det inte är “jag orkade inte”, “jag

glömde bort det”, eller “jag glömde bort att jag behövde fixa något annat” så är vi okej med det.

Vi siktar på att alltid arbeta tillsammans, om vi inte har en *riktigt bra* anledning att göra ett undantag. Detta för att vi båda ska kunna ge input på vad vi skriver, och att båda ska hänga med i utvecklingsprocessen. Vi parprogrammerar alltid (dvs. tittar på, pratar om, och skriver) samma kodsnuitt. Vi har hittills alltid jobbat tillsammans.

Om Fritiof kallar sig själv gammal igen bryter vi dock **alla** labbgrupper.

(Resten av dokumentet ska inte lämnas in förrän projektet är klart, men **titta ändå genom allt för att se vilka delar ni behöver arbeta med och fylla i kontinuerligt under projektets gång!**)

Projektrapport

Även om denna del inte ska lämnas in förrän projektet är klart, är det **viktigt att arbeta med den kontinuerligt** under projektets gång! Speciellt finns det några avsnitt där ni ska beskriva information som ni lätt kan glömma av när veckorna går (vilket flera tidigare studenter också har kommenterat).

Tänk på att ligga på lagom ambitionsnivå! En välskriven implementationsbeskrivning (avsnitt 6) hamnar normalt på **3-6 sidor** i det givna formatet och radavståndet, med ett par mindre UML-diagram och kanske ett par andra små illustrerande bilder vid behov. Hela projektrapporten (denna sista halva av dokumentet) behöver sällan mer än 10-12 sidor.

6. Implementationsbeskrivning

I det här avsnittet, och dess underavsnitt (6.x), beskriver ni olika aspekter av själva *implementationen*, under förutsättning att läsaren redan förstår vad *syftet* med projektet är (det har ju beskrivits tidigare).

Tänk er att någon ska vidareutveckla projektet, kanske genom att fixa eventuella buggar eller skapa utökningar. Då finns det en hel del som den personen kan behöva förstå så att man vet *var* funktionaliteten finns, *hur* den är uppdelad, och så vidare. Algoritmer och övergripande design passar också in i det här kapitlet.

Bilder, flödesdiagram, osv. är starkt rekommenderat!

Skapa gärna egna delkapitel för enskilda delar, om det underlättar. **Ta inte bort några rubriker!**

Även detta är en del av examinationen som visar att ni förstår vad ni gör!

6.1. Milstolpar

Ange för varje milstolpe om ni har genomfört den helt, delvis eller inte alls.

Detta är till för att labbhandledaren ska veta vilken funktionalitet man kan "leta efter" i koden. Själva bedömningen beror *inte* på antalet milstolpar i sig, och inte heller på om man "hann med" milstolparna eller inte!

6.2. Dokumentation för programstruktur, med UML-diagram

Programkod behöver dokumenteras för att man ska förstå hur den fungerar och hur allt hänger ihop. Vissa typer av dokumentation är direkt relaterad till ett enda fält, en enda metod eller en enda klass och placeras då lämpligast vid fältet, metoden eller klassen i en Javadoc-kommentar, *inte här*. Då är det både enklare att hitta dokumentationen och större chans att den faktiskt uppdateras när det sker ändringar. Annan dokumentation är mer övergripande och saknar en naturlig plats i koden. Då kan den placeras här. Det kan gälla till exempel:

- **Övergripande programstruktur**, t.ex. att man har implementerat ett spel som styrs av timer-tick n gånger per sekund där man vid varje sådant tick först tar hand om input och gör eventuella förflyttningar för objekt av typ X, Y och Z, därefter kontrollerar kollisioner vilket sker med hjälp av klass W, och till slut uppdaterar skärmen.
- **Översikter över relaterade klasser** och hur de hänger ihop.
 - o Här kan det ofta vara bra att använda **UML-diagram** för att illustrera – det finns även i betygskraven. Fundera då först på vilka grupper av klasser det är ni vill beskriva, och skapa sedan ett UML-diagram för varje grupp av klasser.
 - o Notera att det sällan är särskilt användbart att lägga in hela projektet i ett enda gigantiskt diagram (vad är det då man fokuserar på?). Hitta intressanta delstrukturer och visa dem. Ni behöver normalt inte ha med fält eller metoder i diagrammen.
 - o **Skriv sedan en textbeskrivning av vad det är ni illustrerar med UML-diagrammet.** Texten är den huvudsakliga dokumentationen medan UML-diagrammet hjälper läsaren att förstå texten och få en översikt.
 - o IDEA kan hjälpa till att göra klassdiagram som ni sedan kan klippa och klistra in i dokumentet. Högerklicka i en editor och välj Diagrams / Show Diagram. Ni kan sedan lägga till och ta bort klasser med högerklicksmenyn. Exportera till bildfil med högerklick / Export to File.

I det här avsnittet har ni också en möjlighet att visa upp era kunskaper genom att diskutera koden i objektorienterade termer. Ni kan till exempel diskutera hur ni använder och har nytta av (åtminstone en del av) objekt/klasser, konstruktörer, typhierarkier, interface, ärvning, overriding, abstrakta klasser, subtypspolymorfism, och inkapsling (accessnivåer).

Labbandledaren och examinatorn kommer bland annat att använda dokumentationen i det här avsnittet för att förstå programmet vid bedömningen. Ni kan också tänka er att ni själva ska vidareutveckla projektet efter att en annan grupp har utvecklat grunden. Vad skulle ni själva vilja veta i det läget?

När ni pratar om klasser och metoder ska deras namn anges tydligt (inte bara "vår timerklass" eller "utritningsmetoden").

Framhäv gärna det ni själva tycker är **bra/intressanta lösningar** eller annat som handledaren borde titta på vid den senare genomgången av programkoden.

Vi räknar med att de flesta projekt behöver runt **3-6 sidor** för det här avsnittet.

7. Användarmanual

När ni har implementerat ett program krävs det också en manual som förklarar hur programmet fungerar. Ni ska beskriva programmet tillräckligt mycket för att en labbandledare själv ska kunna *starta det, testa det och förstå hur det används*.

Inkludera flera (**minst 3**) **skärmdumpar** som visar hur programmet ser ut! Dessa ska vara "inline" i detta dokument, inte i separata filer. Sikta på att visa de relevanta delarna av programmet för någon som *inte* startar det själv, utan bara läser manualen!

(Glöm inte att ta bort våra instruktioner, och exportera till PDF-format med korrekt

namn enligt websidorna, innan ni skickar in!)